MULTICLASS IRIS FLOWER CLASSIFICATION

A Comparative Study of One-vs-Rest and Softmax Logistic Regression

PROJECT REPORT

Submitted in partial fulfilment of the requirements for

Bachelor of Technology

Computer Science and Engineering – Artificial Intelligence and Machine Learning

Submitted by: Nitya Jaiswal

Roll Number: 2402841530051

Under the Guidance of: Mr. Gaurav Narain Singh

United Institute of Technology, Naini, Prayagraj

Dr. A.P.J. Abdul Kalam Technical University (AKTU), Uttar Pradesh

Academic Session: 2026–2027

DECLARATION

I hereby declare that the project entitled **"Multiclass Iris Flower Classification: A Comparative Study of One-vs-Rest and Softmax Logistic Regression"** is an original work carried out by me as part of the academic requirements of the Bachelor of Technology programme in Computer Science and Engineering (Artificial Intelligence and Machine Learning).

The project has been implemented using Python and machine learning libraries. The results presented in this report are based on the experiments conducted during the development of the project.

I further declare that this work has not been submitted elsewhere for the award of any other degree or qualification.

Student Name: Nitya Jaiswal

Roll Number: 2402841530051

Date: 05 November 2026

Signature: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members and the Department of Computer Science and Engineering for their guidance and support throughout the development of this project.

I would also like to acknowledge the developers and contributors of Python, Scikit-learn, NumPy, Pandas, Matplotlib and Seaborn, whose open-source tools made the implementation and analysis of this machine learning project possible.

Finally, I thank my institution for providing the academic environment and resources required to complete this work.

ABSTRACT

Multiclass classification is an important task in machine learning in which an algorithm assigns an input observation to one of several predefined categories.

This project presents a comparative study of two approaches for multiclass Iris flower classification: One-vs-Rest (OvR) Logistic Regression and Softmax Logistic Regression.

The original Iris dataset available through Scikit-learn is used. It contains 150 observations belonging to three species: Setosa, Versicolor and Virginica. Each observation contains four numerical features: sepal length, sepal width, petal length and petal width.

The project involves dataset preparation, exploratory data analysis, preprocessing, model implementation, hyperparameter tuning, evaluation and interactive prediction. Both classifiers are implemented using Scikit-learn pipelines with StandardScaler to standardize the input features. The project also includes a separate L-BFGS-B loss-convergence diagnostic and class-wise coefficient analysis to interpret the fitted models.

Hyperparameter optimization is performed using GridSearchCV with five-fold Stratified Cross-Validation and macro F1-score as the scoring metric.

On the held-out test set containing 30 observations, the OvR model achieved an accuracy of 93.33%, whereas the Softmax model achieved an accuracy of 100%. Their macro ROC-AUC scores were 99.33% and 100%, respectively.

The results indicate that Softmax Logistic Regression performed better on the selected test split. The project also provides an interactive command-line prediction system and automated tests to validate the implementation.

Keywords: Machine Learning, Multiclass Classification, Logistic Regression, One-vs-Rest, Softmax, Iris Dataset, Scikit-learn.

TABLE OF CONTENTS

1. Introduction

2. Dataset Description and Exploratory Data Analysis

3. Data Preprocessing

4. Methodology and Algorithm Design

5. Implementation and Hyperparameter Optimization

6. Results and Performance Evaluation

7. Interactive Prediction System

8. Testing and Validation

9. Conclusion and Future Scope

10. References

11. Appendix

LIST OF FIGURES AND TABLES

List of Figures

1. Iris sepal length distribution histogram.

2. Iris sepal width distribution histogram.

3. Iris petal length distribution histogram.

4. Iris petal width distribution histogram.

5. Sepal length boxplot by species.

6. Sepal width boxplot by species.

7. Petal length boxplot by species.

8. Petal width boxplot by species.

9. Pairwise relationships among Iris features.

10. Feature correlation heatmap.

11. Iris species distribution.

12. OvR confusion matrix.

13. Softmax confusion matrix.

14. Multiclass ROC curves.

15. Multiclass precision-recall curves.

16. OvR decision boundary using sepal features.

17. Softmax decision boundary using sepal features.

18. OvR decision boundary using petal features.

19. Softmax decision boundary using petal features.

20. OvR versus Softmax loss convergence.

21. Mean absolute feature coefficient comparison.

22. Class-wise OvR and Softmax coefficient heatmaps.

List of Tables

1. Iris dataset attributes and data types.

2. Iris target class distribution.

3. Dataset descriptive statistics.

4. Train-test partition summary.

5. Preprocessing configuration.

6. OvR and Softmax methodology comparison.

7. Hyperparameter search configuration.

8. Best hyperparameter results.

9. Test-set performance comparison.

10. Class-wise classification report for OvR.

11. Class-wise classification report for Softmax.

12. Loss convergence diagnostic results.

13. Mean absolute coefficient comparison.

14. Automated test cases and validation results.

15. Sample flower measurements and predicted species probabilities.

CHAPTER 1: INTRODUCTION

1.1 Background

Machine Learning is a branch of Artificial Intelligence that enables computer systems to learn patterns from data and make predictions without being explicitly programmed for every individual task.

Classification is a supervised learning technique in which a model learns from labelled examples and predicts the category of new observations.

Binary classification involves two possible classes, whereas multiclass classification involves three or more classes.

The Iris flower classification problem is a widely used example of multiclass classification. It involves identifying the species of an Iris flower using measurements of its physical characteristics.

1.2 Problem Statement

The objective of this project is to develop a machine learning system capable of classifying Iris flowers into one of three species using four numerical measurements.

The project focuses on comparing two multiclass Logistic Regression approaches:

• One-vs-Rest Logistic Regression

• Softmax Logistic Regression

The comparison is performed using standard classification metrics and visual evaluation techniques.

1.3 Objectives

The primary objectives are:

1. Understand the original Iris dataset.

2. Perform exploratory data analysis.

3. Prepare the dataset for machine learning.

4. Implement One-vs-Rest Logistic Regression.

5. Implement Softmax Logistic Regression.

6. Apply feature standardization.

7. Perform hyperparameter tuning using GridSearchCV.

8. Evaluate both models using classification metrics.

9. Visualize decision boundaries and model performance.

10. Develop an interactive prediction system.

11. Validate the implementation through automated testing.

1.4 Scope

The project is designed as a beginner-friendly but technically sound machine learning application.

It focuses on supervised multiclass classification using a structured numerical dataset.

The implementation includes data analysis, model training, optimization, evaluation and command-line prediction.

1.5 Software and Technologies Used

Technology	Purpose
Python 3.14.7	Programming language
VS Code	Development environment
Jupyter Notebook	Exploratory analysis
NumPy	Numerical operations
Pandas	Data manipulation
Matplotlib	Data visualization
Seaborn	Statistical visualization
Scikit-learn	Machine learning
Joblib	Model serialization
Pytest	Automated testing

CHAPTER 2: DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS

2.1 Dataset Overview

The project uses the original Iris dataset provided by Scikit-learn.

The dataset contains 150 observations distributed equally among three Iris species.

Property	Description
Total observations	150
Numerical features	4
Target classes	3
Setosa samples	50
Versicolor samples	50
Virginica samples	50
Missing values	0

2.2 Input Features

The dataset contains the following four numerical features:

1. Sepal Length (cm)

2. Sepal Width (cm)

3. Petal Length (cm)

4. Petal Width (cm)

The target variable represents the Iris species.

2.3 Target Classes

The three target categories are:

• Setosa

• Versicolor

• Virginica

The dataset is balanced, with 50 observations belonging to each category.

2.4 Dataset Preparation

The original dataset was loaded using the `load_iris(as_frame=True)` function from Scikit-learn.

The following preparation operations were performed:

• Renaming feature columns into snake_case format.

• Creating a unique sample identifier.

• Mapping numerical target values to species names.

• Organizing the dataset into a structured Pandas DataFrame.

• Exporting the prepared dataset into CSV format.

The resulting dataset is stored as:

`data/iris_botanical_records.csv`

It contains 150 rows and 6 columns, including the sample identifier, four features and target category.

2.5 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the dataset's structure, distributions and relationships.

The analysis included:

• Dataset summary and descriptive statistics.

• Feature distributions using histograms.

• Outlier analysis using boxplots.

• Feature relationships using pairplots.

• Correlation analysis using a heatmap.

• Species distribution analysis.

2.6 Statistical Observations

The overall feature statistics obtained during analysis are:

Feature	Mean	Median	Standard Deviation
Sepal Length	5.8433	5.80	0.8281
Sepal Width	3.0573	3.00	0.4359
Petal Length	3.7580	4.35	1.7653
Petal Width	1.1993	1.30	0.7622

The mean petal lengths across the three species are:

Species	Mean Petal Length
Setosa	1.462 cm
Versicolor	4.260 cm
Virginica	5.552 cm

These observations indicate that petal measurements provide useful information for distinguishing the species.

2.7 Data Quality Findings

The dataset analysis identified:

• No missing values.

• One duplicate feature observation.

• Four IQR-based outliers in sepal width.

• No IQR-based outliers in the other three features.

• Equal class distribution.

The identified duplicate observation and outliers were retained in the dataset.

2.8 EDA Visualizations

The project generates the following visualizations:

• Feature histograms.

• Feature boxplots.

• Pairplot.

• Correlation heatmap.

• Species distribution chart.

These visualizations are stored in `outputs/figures/`.

CHAPTER 3: DATA PREPROCESSING

3.1 Feature and Target Separation

The four numerical measurements are selected as independent variables (X), while the species category is selected as the dependent variable (y).

The sample identifier is not used as a predictive feature.

3.2 Train-Test Split

The dataset is divided into training and testing subsets.

An 80:20 split is used.

Dataset	Observations
Training set	120
Testing set	30
Total	150

Stratified sampling is applied to preserve the class distribution in both subsets.

The random state is set to 42 to ensure reproducibility.

3.3 Feature Standardization

The numerical features have different ranges and scales.

StandardScaler is used to standardize the features by removing the mean and scaling to unit variance.

The standardization operation is incorporated inside the machine learning Pipeline.

This ensures that the scaler is fitted only on the training data during model training and within individual cross-validation folds, preventing data leakage.

3.4 Preprocessing Configuration

Parameter	Value
Test size	20%
Training size	80%
Random state	42
Stratification	Enabled
Feature scaling	StandardScaler

CHAPTER 4: METHODOLOGY AND ALGORITHM DESIGN

4.1 Logistic Regression

Logistic Regression is a supervised learning algorithm commonly used for classification.

Although its name contains the term regression, it is used to estimate class probabilities and assign observations to categories.

For binary classification, the sigmoid function is commonly used to map a linear score to a probability.

4.1.1 Sigmoid Function for Binary Logistic Regression

For a feature vector x, the linear score is z = w^T x + b. The sigmoid function converts this score into a probability:

P(y = 1 | x) = 1 / (1 + exp(-z))

The predicted probability lies between 0 and 1. A classification threshold is then used to assign a binary class.

4.1.2 Binary Cross-Entropy Loss

For N observations with binary labels y_i and predicted probabilities p_i, the binary cross-entropy objective is:

L_binary = -(1/N) * sum[i=1 to N] { y_i log(p_i) + (1-y_i) log(1-p_i) }

An L2 regularization term penalizes large feature weights. In the diagnostic convergence experiment, the objective is L_binary + (lambda/2) ||*w*||^2, *where lambda = 1/(C*N) for the mean-loss objective used by the L-BFGS diagnostic. The intercept is not penalized.

4.2 One-vs-Rest Classification

One-vs-Rest is a strategy used to extend binary classifiers to multiclass classification problems.

For a dataset containing three classes, OvR trains three separate binary classifiers.

Each classifier learns to distinguish one class from all remaining classes.

In this project:

• Classifier 1: Setosa versus the remaining classes.

• Classifier 2: Versicolor versus the remaining classes.

• Classifier 3: Virginica versus the remaining classes.

For K classes, OvR trains K binary classifiers. For class k, the binary target is y_k = 1 when the observation belongs to class k and 0 otherwise. Each classifier estimates P(y_k = 1 | x) using its own sigmoid model. The class with the highest decision score is selected as the prediction. The separately returned probability estimates should not be confused with the OvR decision scores used by the classifier for class selection.

The OvR diagnostic objective aggregates the K binary losses:

L_OvR = sum[k=1 to K] L_binary,k

For a new observation, the model produces a score for each class. The class with the highest score is selected as the prediction.

4.3 Softmax Logistic Regression

Softmax Logistic Regression directly handles multiple classes using a single multiclass model.

The Softmax function converts class scores into a probability distribution.

For class k, the probability is:

$P(y = k \mid x) = \exp(z_k) / \Sigma \exp(z_j)$

where:

• z_k represents the score for class k.

• The denominator represents the sum of exponentials of all class scores.

• The resulting probabilities lie between 0 and 1.

• The probabilities across all classes sum to 1.

The class with the highest predicted probability is selected.

For K classes, the class score is $z_k = w_k^T x + b_k$. The Softmax probability is:

$P(y = k \mid x) = \exp(z_k) / \sum_{j=1}^{K} \exp(z_j)$

The categorical cross-entropy objective is:

$L_{softmax} = -(1/N) * \sum_{i=1}^{N} \log(P(y_i \mid x_i))$

With L2 regularization, the diagnostic objective is $L_{softmax} + (\lambda/2) \sum_{k=1}^{K} ||w_k||^2$, where $\lambda = 1/(CN)$. The class biases are not penalized.

4.4 Difference Between OvR and Softmax

Parameter	One-vs-Rest	Softmax
Classification strategy	Multiple binary classifiers	Direct multiclass classification
Number of classifiers	Three	One multiclass model
Class handling	One class against all others	All classes jointly
Output	Class scores/probabilities	Multiclass probability distribution
Implementation	OneVsRestClassifier	LogisticRegression

4.5 Machine Learning Pipelines

Both models use Scikit-learn Pipelines.

The pipeline contains:

1. StandardScaler.

2. Logistic Regression classifier.

For OvR, the Logistic Regression estimator is wrapped inside OneVsRestClassifier.

The use of pipelines provides consistent preprocessing and helps prevent data leakage.

CHAPTER 5: IMPLEMENTATION AND HYPERPARAMETER OPTIMIZATION

5.1 Development Environment

The complete project was developed locally using Visual Studio Code.

Python was used as the primary programming language.

The project is organized into separate modules for configuration, data loading, preprocessing, visualization, model training, hyperparameter tuning, evaluation and prediction.

5.2 Model Training

Two classification pipelines were constructed:

Model 1: OvR Logistic Regression

• StandardScaler

• OneVsRestClassifier

• LogisticRegression

• Solver: lbfgs

• Maximum iterations: 1000

Model 2: Softmax Logistic Regression

• StandardScaler

• LogisticRegression

• Solver: lbfgs

• Maximum iterations: 1000

The initial regularization parameter was C = 1.0.

5.3 Hyperparameter Tuning

GridSearchCV was used to identify suitable regularization parameters.

The search evaluated:

C = [0.01, 0.1, 1, 10, 100]

The tuning configuration was:

Parameter	Value
Search method	GridSearchCV
Cross-validation	StratifiedKFold
Number of folds	5
Scoring metric	Macro F1-score
Shuffle	True

Parameter	Value
Random state	42

The search was performed using the training dataset.

5.4 Hyperparameter Optimization Results

Model	Best C	Best CV Macro F1
OvR	10	95.80%
Softmax	10	96.63%

Softmax achieved a slightly higher cross-validation macro F1-score.

5.5 Model Serialization

The trained and tuned models were saved using Joblib.

Saved model files include:

- `models/ovr_model.joblib`
- `models/softmax_model.joblib`
- `models/ovr_tuned_model.joblib`
- `models/softmax_tuned_model.joblib`

This allows the prediction system to load trained models without retraining them every time.

5.6 Loss Convergence Comparison

The project includes a separate optimization diagnostic to compare how the OvR and Softmax objectives decrease during training. The diagnostic uses the same stratified training partition as the main experiment, fits StandardScaler only on the training data, and uses C = 10, the value selected by the hyperparameter search.

Scikit-learn's LogisticRegression does not expose a complete per-iteration loss history. Therefore, the diagnostic explicitly minimizes the corresponding regularized logistic objectives using the L-BFGS-B optimizer and records the objective after each iteration. This is a transparent convergence diagnostic; it is not presented as the internal iteration log of the Scikit-learn estimator.

Diagnostic	OvR	Softmax
Optimizer	L-BFGS-B	L-BFGS-B
Iterations	22, 17 and 22 for the three binary classifiers	41
Optimizer status	All three successful	Successful
Final objective	0.57475 (aggregate binary loss)	0.09249 (categorical loss)

The diagnostic uses the mean-loss L2 scaling lambda = 1/(C*N), with N equal to the 120 training observations. The OvR objective is the sum of three binary cross-entropy objectives, while Softmax uses categorical cross-entropy. Their final numerical loss magnitudes are therefore not directly comparable as identical quantities. The plot is used to examine convergence behaviour and the reduction in each model's own objective.

The figure is saved as `outputs/figures/ovr_vs_softmax_loss_convergence.png`, and the iteration history is saved as `models/loss_convergence_history.csv`.

CHAPTER 6: RESULTS AND PERFORMANCE EVALUATION

6.1 Evaluation Method

The tuned models were evaluated using the held-out test set containing 30 observations.

The evaluation included:

• Accuracy.

• Precision.

• Recall.

• F1-score.

• Macro ROC-AUC.

• Confusion matrices.

• Multiclass ROC curves.

• Precision-Recall curves.

6.2 Evaluation Metrics

Accuracy: Measures the proportion of correctly classified observations.

Accuracy = Correct Predictions / Total Predictions

Precision: Measures the proportion of predicted positive observations that are actually positive.

Recall: Measures the proportion of actual positive observations correctly identified.

F1-score: Represents the harmonic mean of precision and recall.

ROC-AUC: Measures the model's ability to distinguish between classes using predicted scores.

Macro averaging calculates the metric independently for each class and then takes the unweighted average.

6.3 Test Set Results

The actual evaluation results are:

Metric	OvR	Softmax
Accuracy	93.33%	100.00%
Macro Precision	93.33%	100.00%
Weighted Precision	93.33%	100.00%
Macro Recall	93.33%	100.00%
Weighted Recall	93.33%	100.00%
Macro F1-score	93.33%	100.00%
Weighted F1-score	93.33%	100.00%
Macro ROC-AUC (OvR)	99.33%	100.00%

6.4 Class-wise Classification Results

OvR Logistic Regression

Species	Precision	Recall	F1-score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	0.90	0.90	0.90	10
Virginica	0.90	0.90	0.90	10

Softmax Logistic Regression

Species	Precision	Recall	F1-score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	1.00	1.00	1.00	10
Virginica	1.00	1.00	1.00	10

6.5 Confusion Matrix Analysis

A confusion matrix compares actual class labels with predicted class labels.

The OvR model correctly classified 28 out of 30 test observations.

The Softmax model correctly classified all 30 test observations in the selected test split.

The confusion matrices are saved in:

- `outputs/figures/ovr_confusion_matrix.png`
- `outputs/figures/softmax_confusion_matrix.png`

6.6 ROC and Precision-Recall Analysis

Multiclass ROC curves were generated using a One-vs-Rest evaluation approach.

Precision-Recall curves were also generated to visualize the relationship between precision and recall across classification thresholds.

The figures are saved as:

- `outputs/figures/multiclass_roc_curves.png`
- `outputs/figures/multiclass_precision_recall.png`

6.7 Decision Boundary Visualization

Decision boundaries were generated using two-feature visualization models.

Two feature combinations were considered:

1. Sepal Length and Sepal Width.

2. Petal Length and Petal Width.

Both OvR and Softmax approaches were visualized for each combination.

The resulting figures are stored in `outputs/figures/`.

These visualization-only models were fitted using the complete dataset for plotting purposes. They are separate from the primary four-feature models used for held-out test evaluation.

6.8 Comparative Analysis

The Softmax model achieved higher observed performance than the OvR model on the selected test split.

Its test accuracy was 100%, compared with 93.33% for OvR.

It also achieved a slightly higher five-fold cross-validation macro F1-score.

However, the test dataset contains only 30 observations. Therefore, the results should be interpreted as performance on this particular split rather than proof of perfect performance on unseen real-world data.

6.9 Feature Coefficient Analysis and Model Insights

The tuned logistic-regression pipelines were examined using their learned coefficients. Since StandardScaler is applied before classification, the coefficients describe the change in a class log-odds value (OvR) or class logit (Softmax) associated with a one-standard-deviation increase in a feature, holding the other features constant.

The mean absolute coefficient magnitudes across the three species are:

Feature	OvR Mean Absolute Coefficient	Softmax Mean Absolute Coefficient
Petal length	3.4275	2.9705
Petal width	3.4724	3.0849
Sepal length	0.7378	1.2006
Sepal width	1.3125	1.1480

Petal length and petal width have larger mean absolute coefficient magnitudes than the sepal measurements in both fitted models. This indicates that the models rely more strongly on petal measurements for separating the species in this dataset. Coefficient magnitude is model-specific and should not be interpreted as a causal effect or as a universal feature-importance measure. The sign of a coefficient indicates the direction of its contribution to a particular class score.

Detailed values are saved in `models/model_coefficients_by_species.csv`, and the mean absolute coefficient summary is saved in `models/feature_coefficient_importance.csv`.

CHAPTER 7: INTERACTIVE PREDICTION SYSTEM

7.1 System Overview

An interactive command-line prediction system was developed to allow users to enter flower measurements and obtain predictions from both trained models.

The system loads the saved tuned models using Joblib.

7.2 User Input

The system accepts four measurements:

1. Sepal Length.

2. Sepal Width.

3. Petal Length.

4. Petal Width.

The input values are validated to ensure they are positive and finite numerical values.

A Pandas DataFrame is created with the correct feature names before prediction.

7.3 Prediction Process

The prediction workflow is:

1. Load the tuned OvR model.

2. Load the tuned Softmax model.

3. Accept four measurements from the user.

4. Validate the measurements.

5. Generate predictions using both models.

6. Calculate class probabilities.

7. Display predicted species and probabilities.

7.4 Sample Prediction

The following sample input was tested:

Feature	Input
Sepal Length	5.1
Sepal Width	3.5
Petal Length	1.4
Petal Width	0.2

Actual output:

OvR Prediction

Species	Probability
Setosa	92.52%
Versicolor	7.48%
Virginica	0.00%

Predicted species: **Setosa**

Softmax Prediction

Species	Probability
Setosa	99.75%
Versicolor	0.25%
Virginica	0.00%

Predicted species: **Setosa**

Both models correctly predicted Setosa for this example.

CHAPTER 8: TESTING AND VALIDATION

8.1 Testing Approach

Pytest was used to perform automated testing of the machine learning project.

The tests verify dataset integrity, preprocessing, model loading, prediction validity and probability calculations.

8.2 Automated Test Cases

The following eleven tests were implemented:

Test	Purpose
Dataset shape and columns	Verify dataset dimensions and required columns
Missing value check	Confirm no missing values
Class distribution	Verify three balanced species classes
Train-test split	Check training and testing dimensions
Model loading	Verify both saved tuned models load
Valid predictions	Confirm predictions belong to valid species
Probability validation	Check probability range and sum
Setosa sample prediction	Verify both models classify the example correctly
Loss convergence diagnostic	Verify both optimization objectives decrease and output files are generated
Feature coefficient analysis	Verify class-wise coefficients and importance outputs
EDA visualization execution	Verify the species distribution visualization can be generated

8.3 Test Execution Results

The test command was:

```
python -m pytest -v
```

Actual result:

11 passed

All eleven automated tests passed in the final validation run. The independent validation run completed without warnings. An earlier local run reported two non-failing Seaborn PendingDeprecationWarning messages. The four saved Joblib models were retrained or tuned and loaded successfully under Scikit-learn 1.9.1, matching the version specified in the project requirements. No Scikit-learn version-compatibility warnings were observed in this matching environment.

This confirms that the tested components behaved as expected under the defined test conditions.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project successfully implemented a multiclass Iris flower classification system using two Logistic Regression approaches: One-vs-Rest and Softmax Logistic Regression.

The work covered the complete machine learning workflow, including dataset preparation, exploratory data analysis, preprocessing, model training, hyperparameter optimization, evaluation, visualization and interactive prediction.

Both models were evaluated on the same held-out test dataset.

The OvR model achieved 93.33% test accuracy, while the Softmax model achieved 100% test accuracy on the selected split.

The Softmax model also obtained a slightly higher cross-validation macro F1-score.

The project demonstrates how different multiclass classification strategies can be implemented and compared using a consistent experimental setup. The optimization diagnostic confirms that both model objectives converge successfully, while coefficient analysis shows that petal measurements have larger mean absolute coefficient magnitudes than sepal measurements in both fitted models.

9.2 Limitations

The project has the following limitations:

• The dataset contains only 150 observations.

• The models were evaluated on a single held-out test split.

• The system is designed for the Iris dataset and its four specific features.

• Performance on other flower datasets has not been evaluated.

• A high score on this small test set does not guarantee equivalent real-world performance.

9.3 Future Scope

Possible future improvements include:

1. Testing additional classification algorithms such as Random Forest, Support Vector Machines and K-Nearest Neighbors.

2. Evaluating models using repeated cross-validation.

3. Exploring feature selection and dimensionality reduction.

4. Building a graphical user interface or web application.

5. Deploying the prediction system as an online service.

6. Evaluating the approach on larger botanical datasets.

7. Adding model monitoring and prediction logging.

CHAPTER 10: REFERENCES

1. R. A. Fisher, "The Use of Multiple Measurements in Taxonomic Problems," Annals of Eugenics, vol. 7, no. 2, pp. 179–188, 1936.

2. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

3. Scikit-learn Developers, "The Iris Dataset," Scikit-learn Documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html

4. Scikit-learn Developers, "Logistic Regression," Scikit-learn Documentation. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

5. Scikit-learn Developers, "OneVsRestClassifier," Scikit-learn Documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html

6. Scikit-learn Developers, "GridSearchCV," Scikit-learn Documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

APPENDIX A: PROJECT DIRECTORY STRUCTURE

The project is organized into the following main directories:

• `data/` — Prepared Iris dataset.

• `notebooks/` — Complete notebook covering EDA, model training, tuning, evaluation and coefficient analysis.

• `src/` — Python source modules.

• `models/` — Trained and tuned models, comparison results, convergence history and coefficient analysis CSV files.

• `outputs/figures/` — EDA and evaluation visualizations.

• `outputs/diagrams/` — Mermaid architecture and workflow diagrams.

• `outputs/reports/` — Generated reports and result files.

• `tests/` — Automated testing files.

• `documentation/` — Academic project documentation.

APPENDIX B: IMPORTANT EXECUTION COMMANDS

Activate the virtual environment:

```
.\.venv\Scripts\Activate.ps1
```

Run exploratory analysis using Jupyter:

```
jupyter notebook
```

Train the models:

```
python -m src.train_model
```

Perform hyperparameter tuning:

```
python -m src.hyperparameter_tuning
```

Evaluate the models:

```
python -m src.evaluate_model
```

Generate decision boundaries:

```
python -m src.decision_boundary
```

Compare OvR and Softmax loss convergence:

```
python -m src.loss_convergence
```

Analyze learned feature coefficients:

```
python -m src.feature_analysis
```

Generate the complete academic report:

```
python -m src.generate_report
```

Run interactive prediction:

```
python -m src.predict
```

Execute automated tests:

```
python -m pytest -v
```

APPENDIX C: CORE IMPLEMENTATION CODE

The following concise implementation excerpts show the main training, tuning, evaluation and prediction workflow. The complete source modules are available in the `src/` directory of the submitted project.

C.1 Model Pipeline Construction

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.pipeline import Pipeline

ovr_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", OneVsRestClassifier(
        LogisticRegression(C=1.0, solver="lbfgs", max_iter=1000)
    ))
])

softmax_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(
        C=1.0, solver="lbfgs", max_iter=1000
    ))
])
```

C.2 Stratified Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

C.3 Hyperparameter Optimization

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = GridSearchCV(
    estimator=model_pipeline,
```

```
    param_grid={"classifier__C": [0.01, 0.1, 1, 10, 100]},
    scoring="f1_macro", cv=cv, n_jobs=-1
)
search.fit(X_train, y_train)
best_model = search.best_estimator_
```

For the OvR pipeline, the corresponding parameter name is `classifier__estimator__C`.

C.4 Model Evaluation

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

y_pred = best_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
macro_f1 = f1_score(y_test, y_pred, average="macro")
confusion = confusion_matrix(y_test, y_pred)
```

C.5 Prediction from Botanical Measurements

```
import pandas as pd

sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=["sepal_length_cm", "sepal_width_cm",
             "petal_length_cm", "petal_width_cm"]
)
predicted_species = best_model.predict(sample)[0]
class_probabilities = best_model.predict_proba(sample)[0]
```

END OF REPORT

APPENDIX D: PROJECT FIGURES

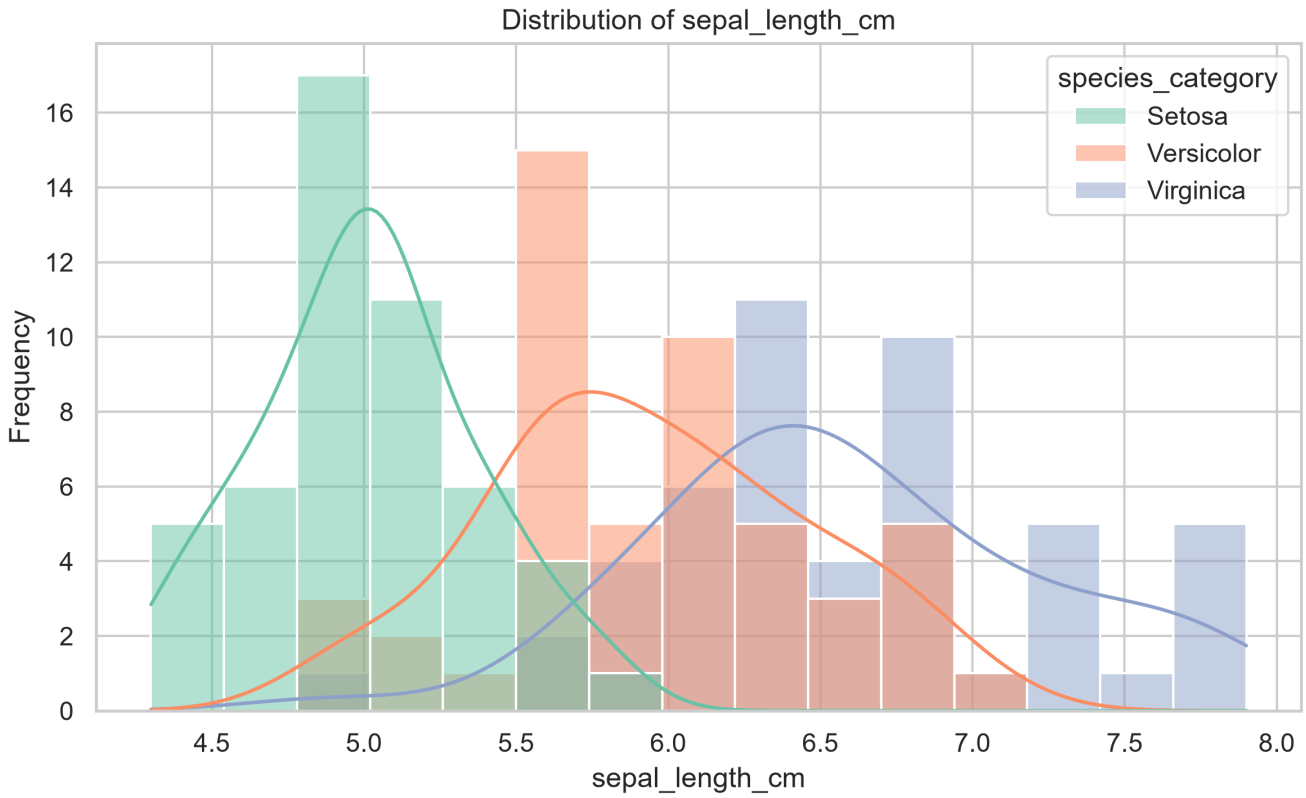


Figure D.1: Distribution of Sepal Length

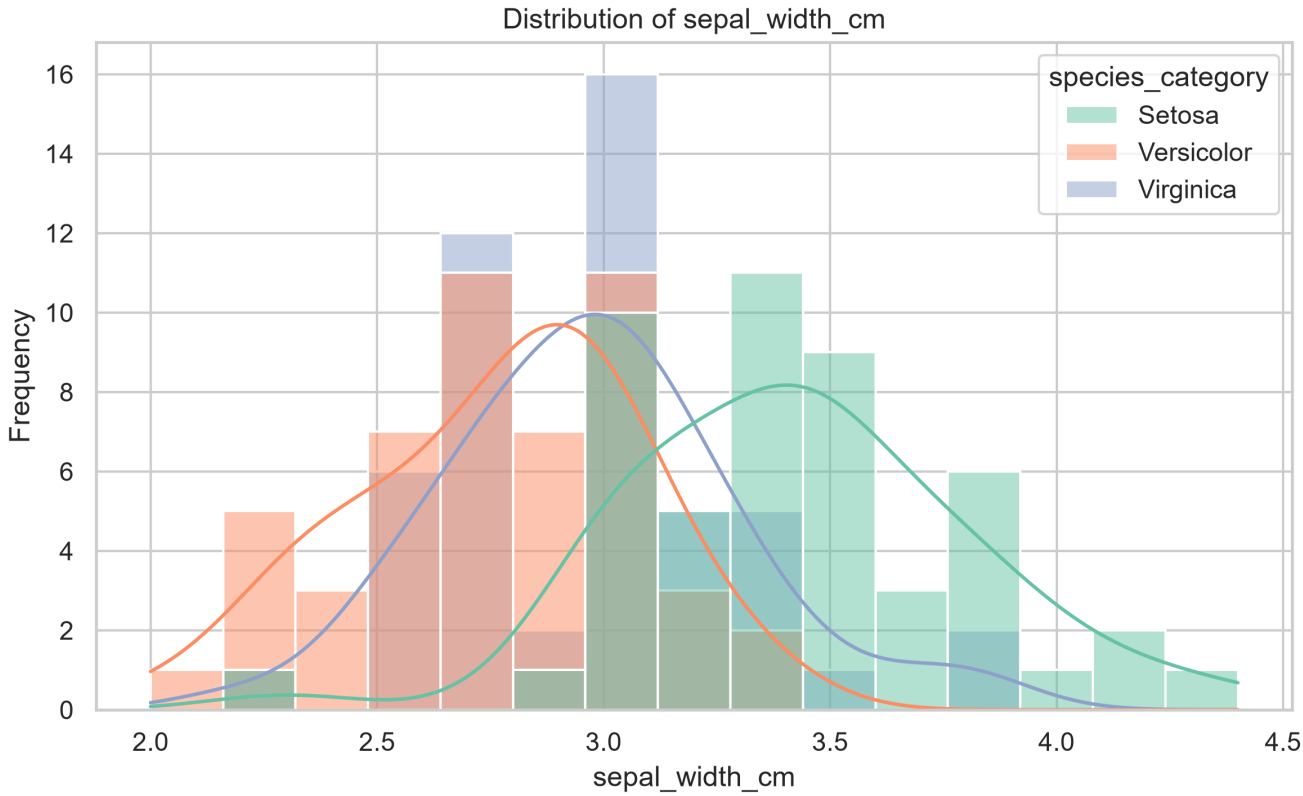


Figure D.2: Distribution of Sepal Width

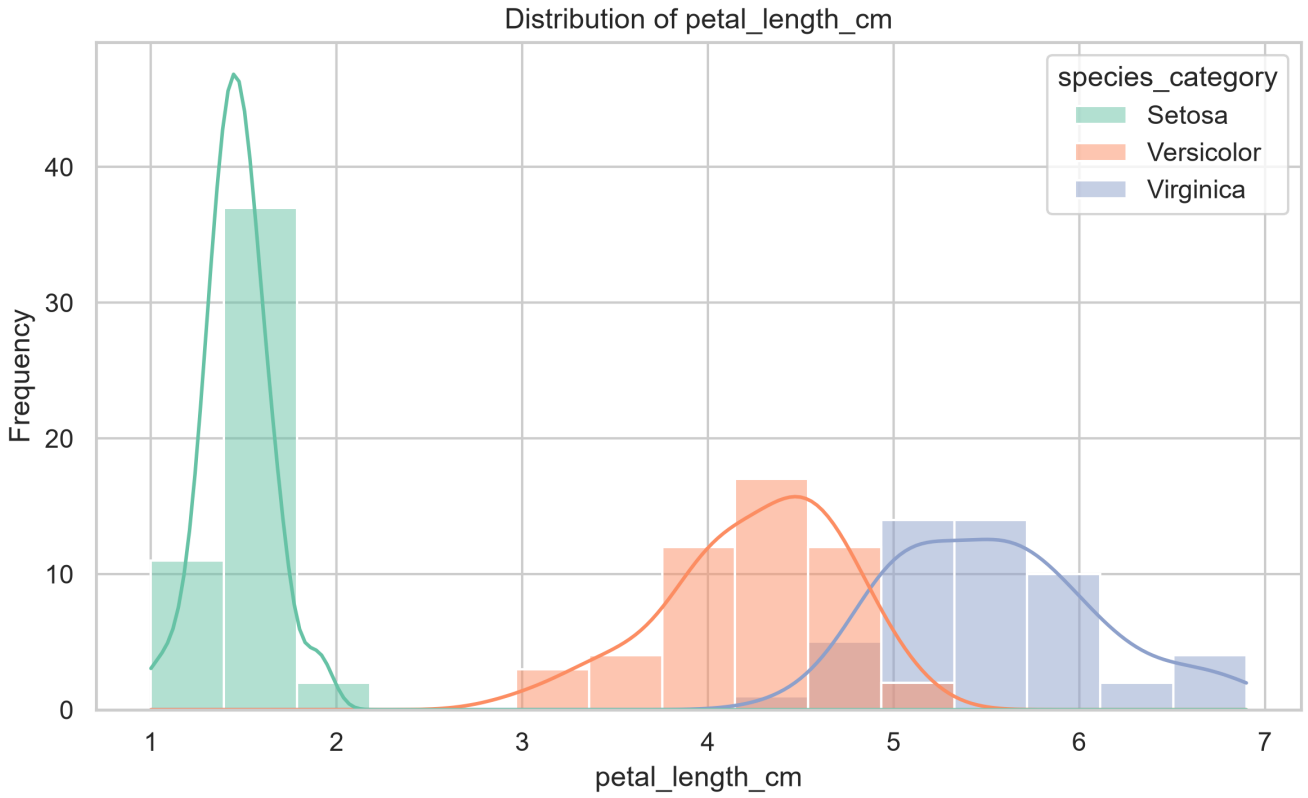


Figure D.3: Distribution of Petal Length

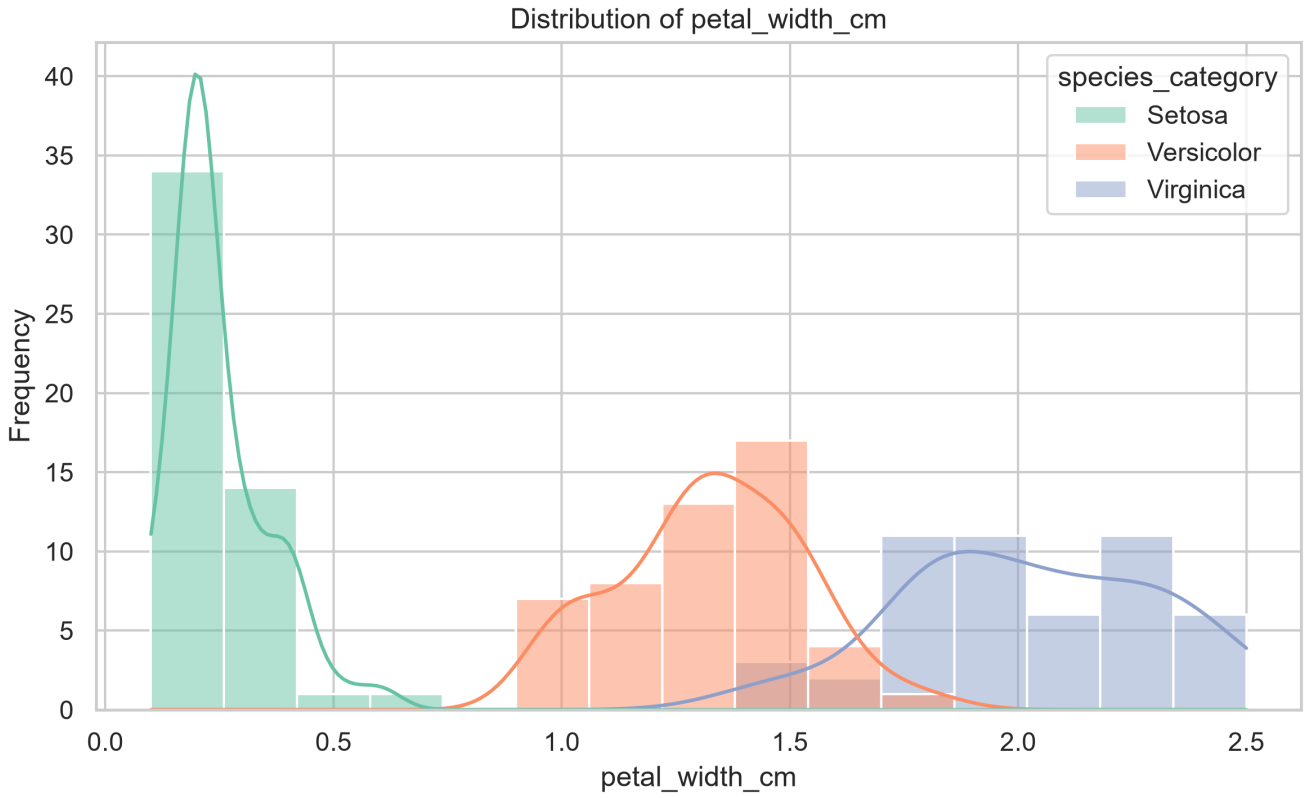


Figure D.4: Distribution of Petal Width

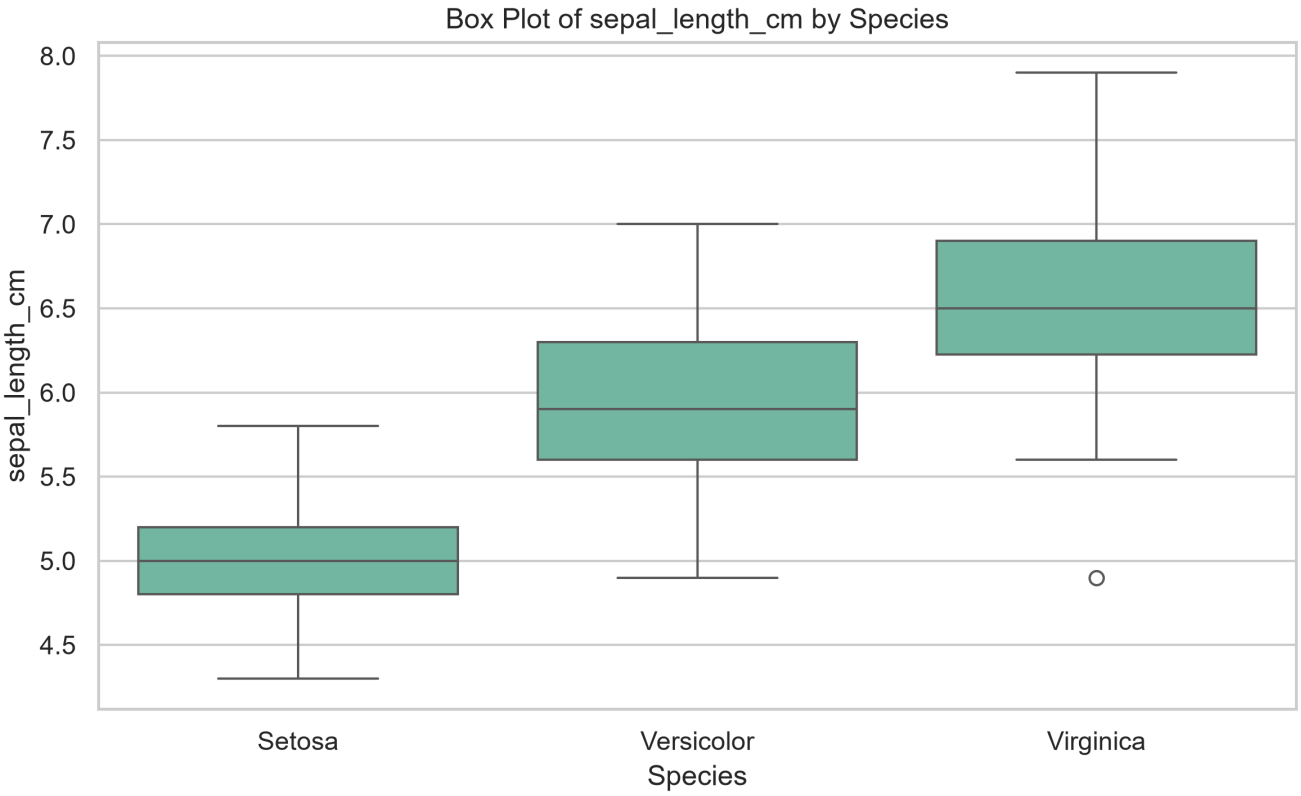


Figure D.5: Boxplot of Sepal Length

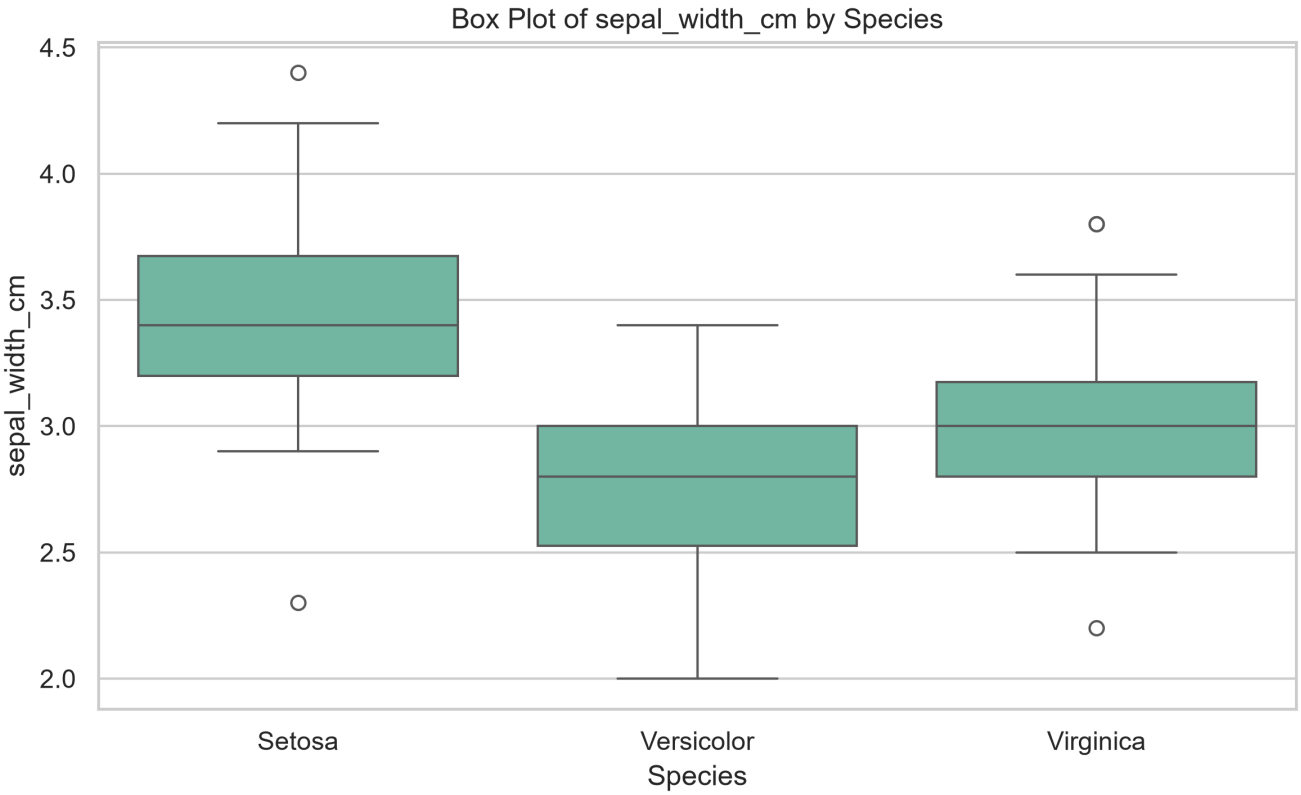


Figure D.6: Boxplot of Sepal Width

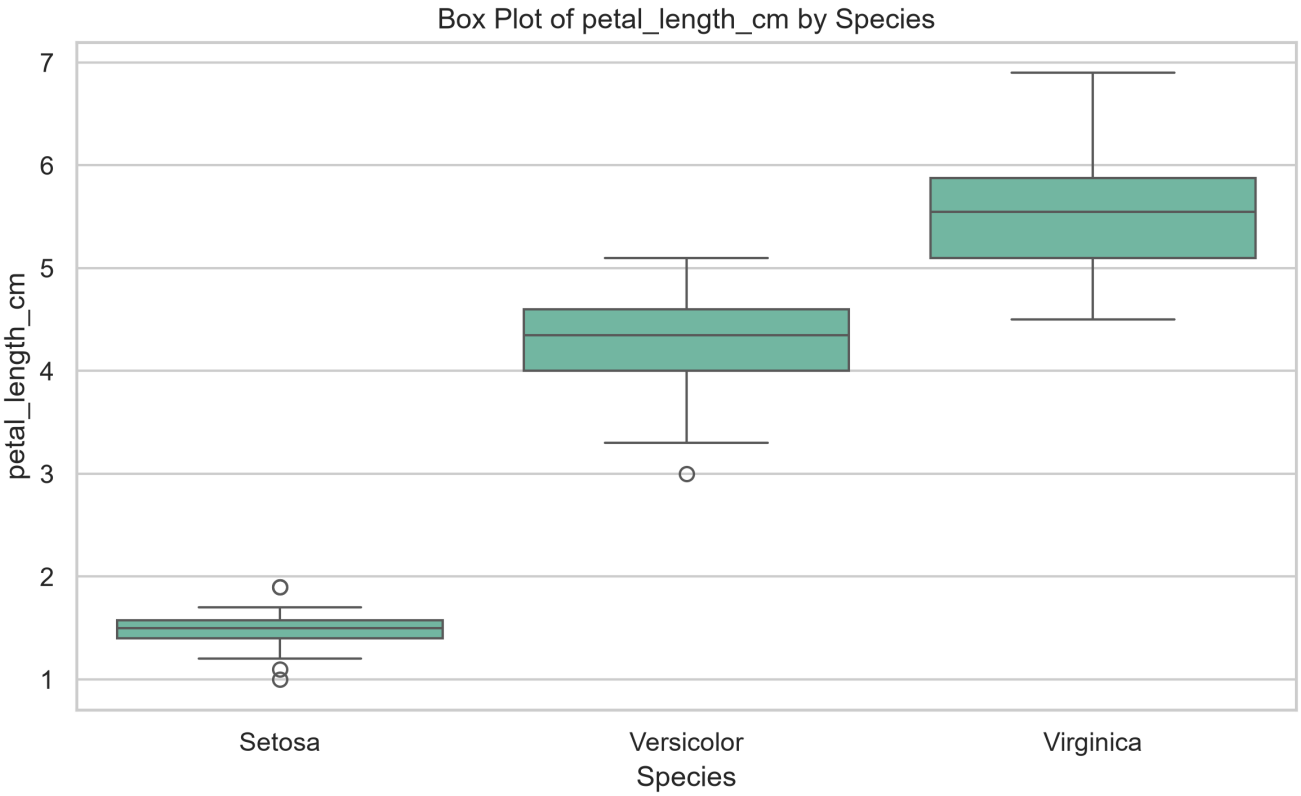


Figure D.7: Boxplot of Petal Length

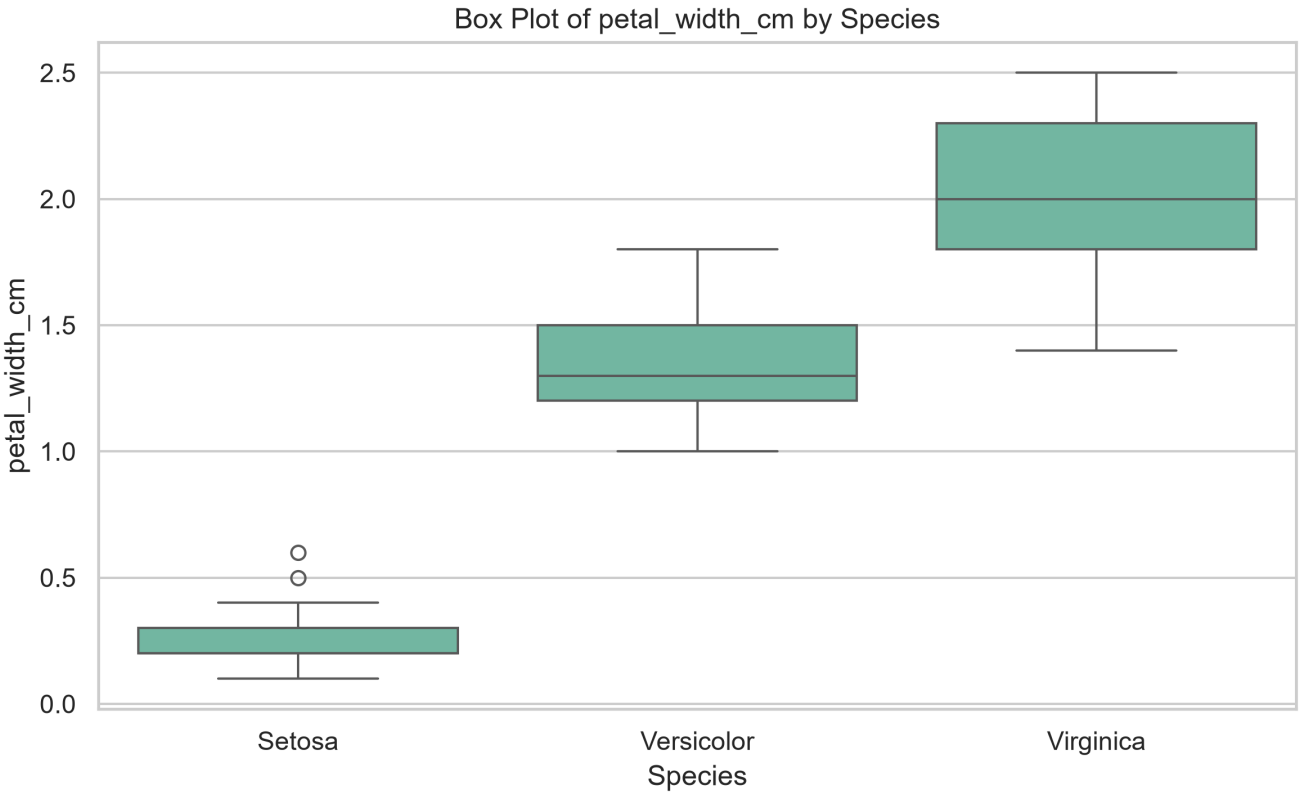


Figure D.8: Boxplot of Petal Width

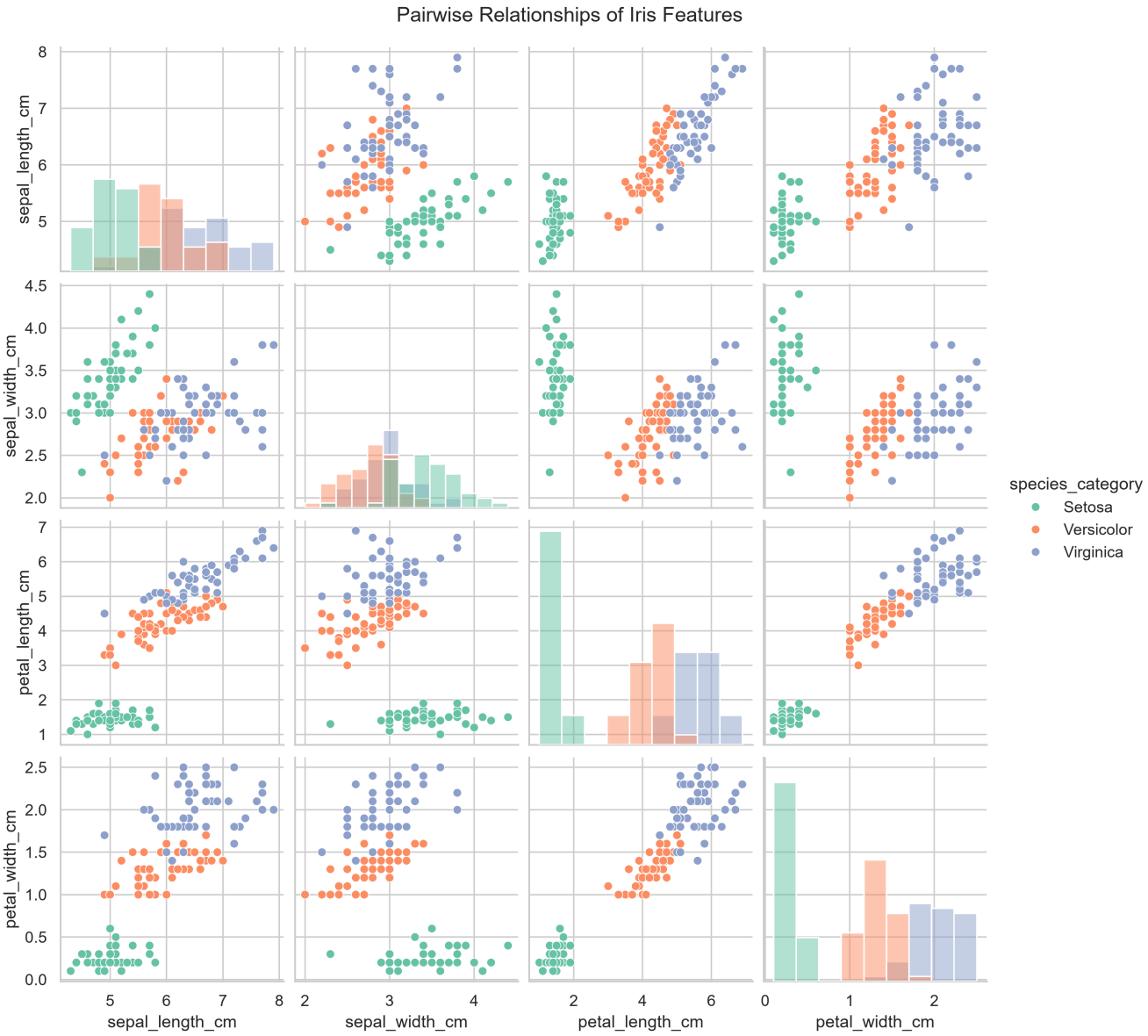


Figure D.9: Pairwise Feature Relationships

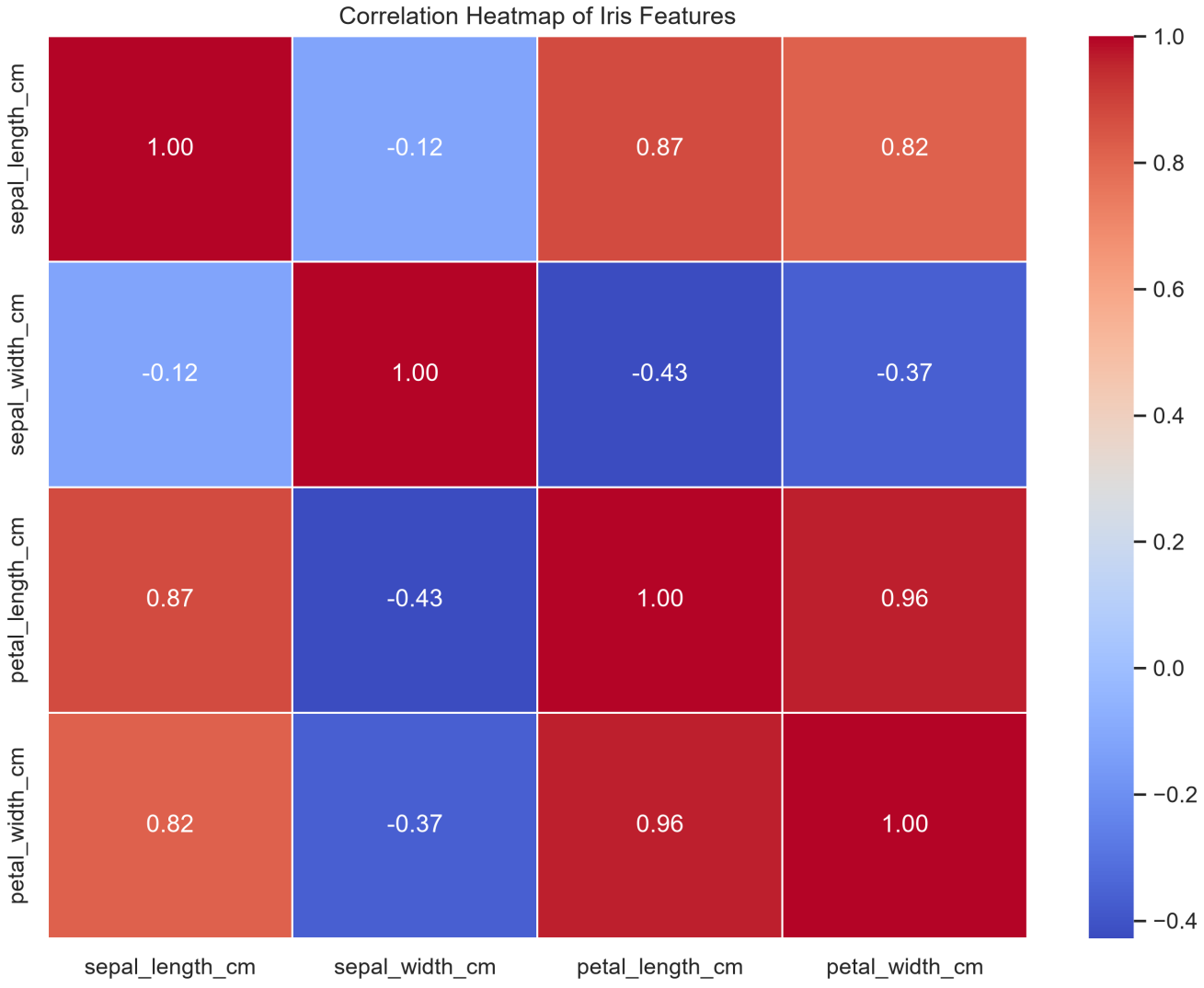


Figure D.10: Feature Correlation Heatmap



Figure D.11: Iris Species Distribution

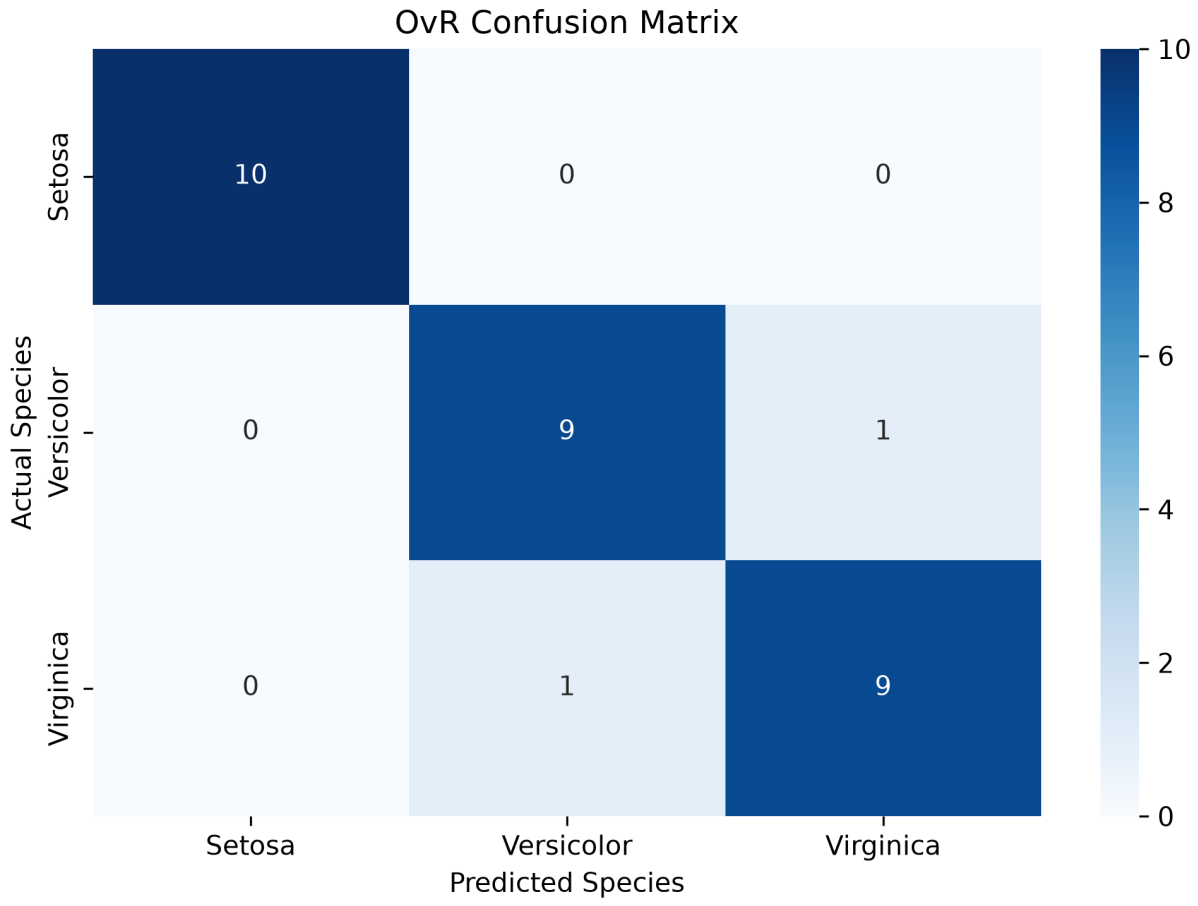


Figure D.12: OvR Confusion Matrix

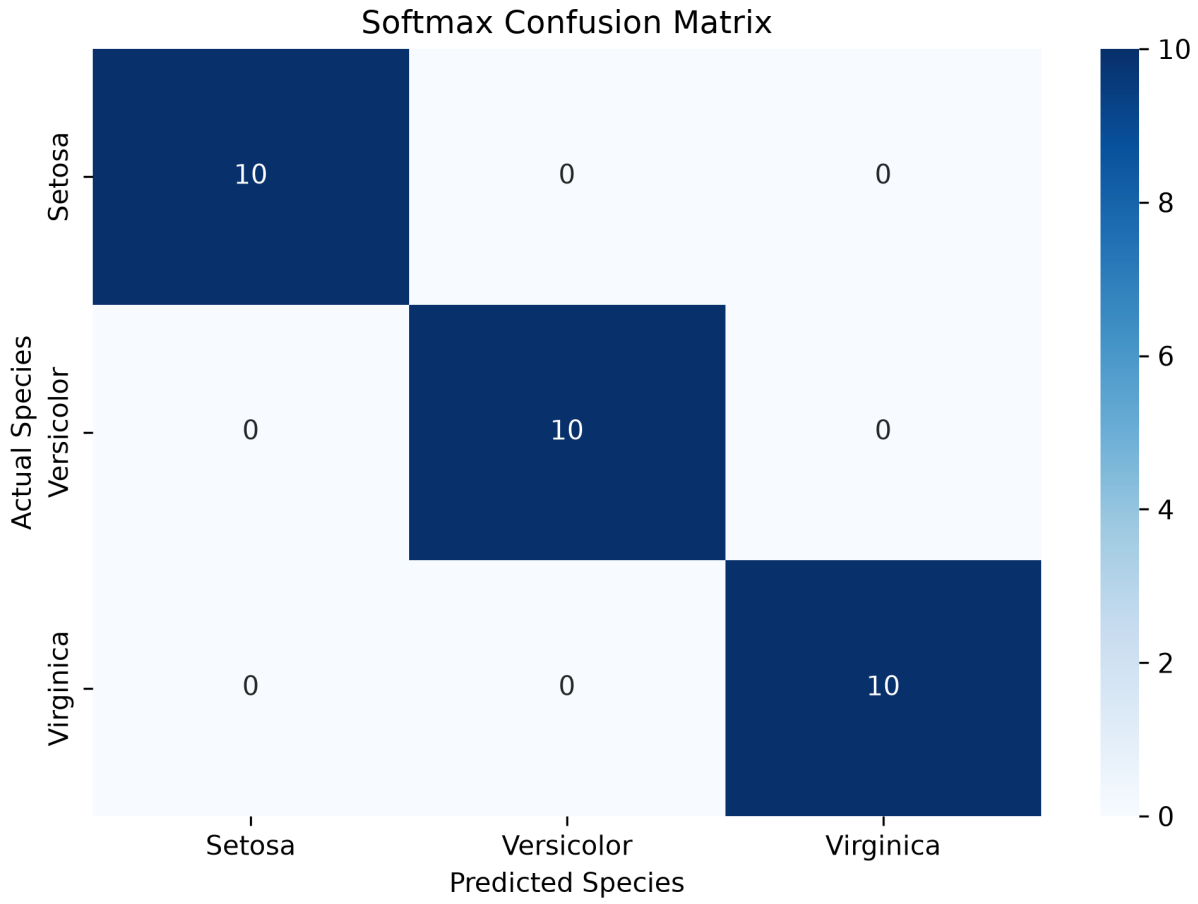


Figure D.13: Softmax Confusion Matrix

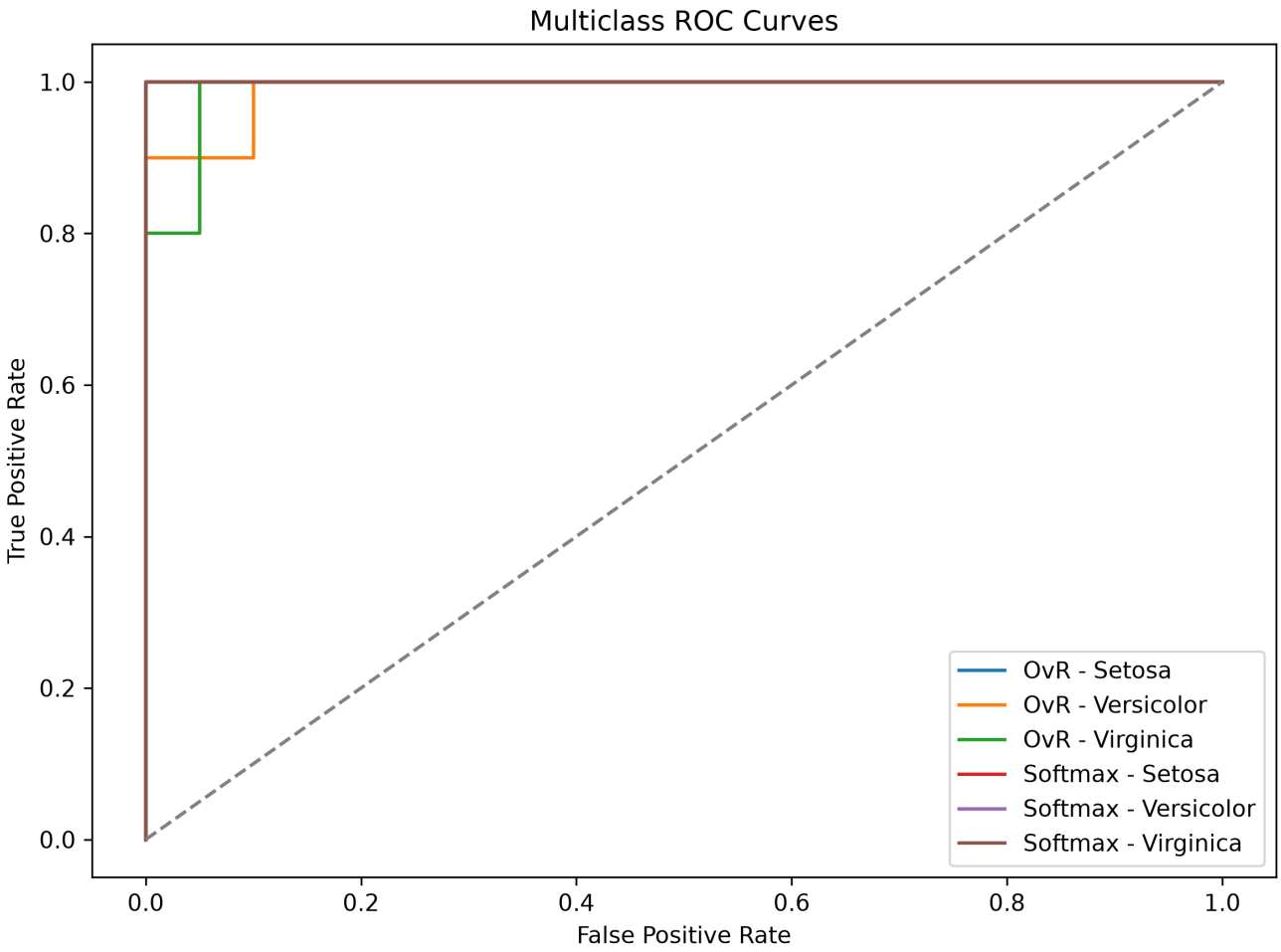


Figure D.14: Multiclass ROC Curves

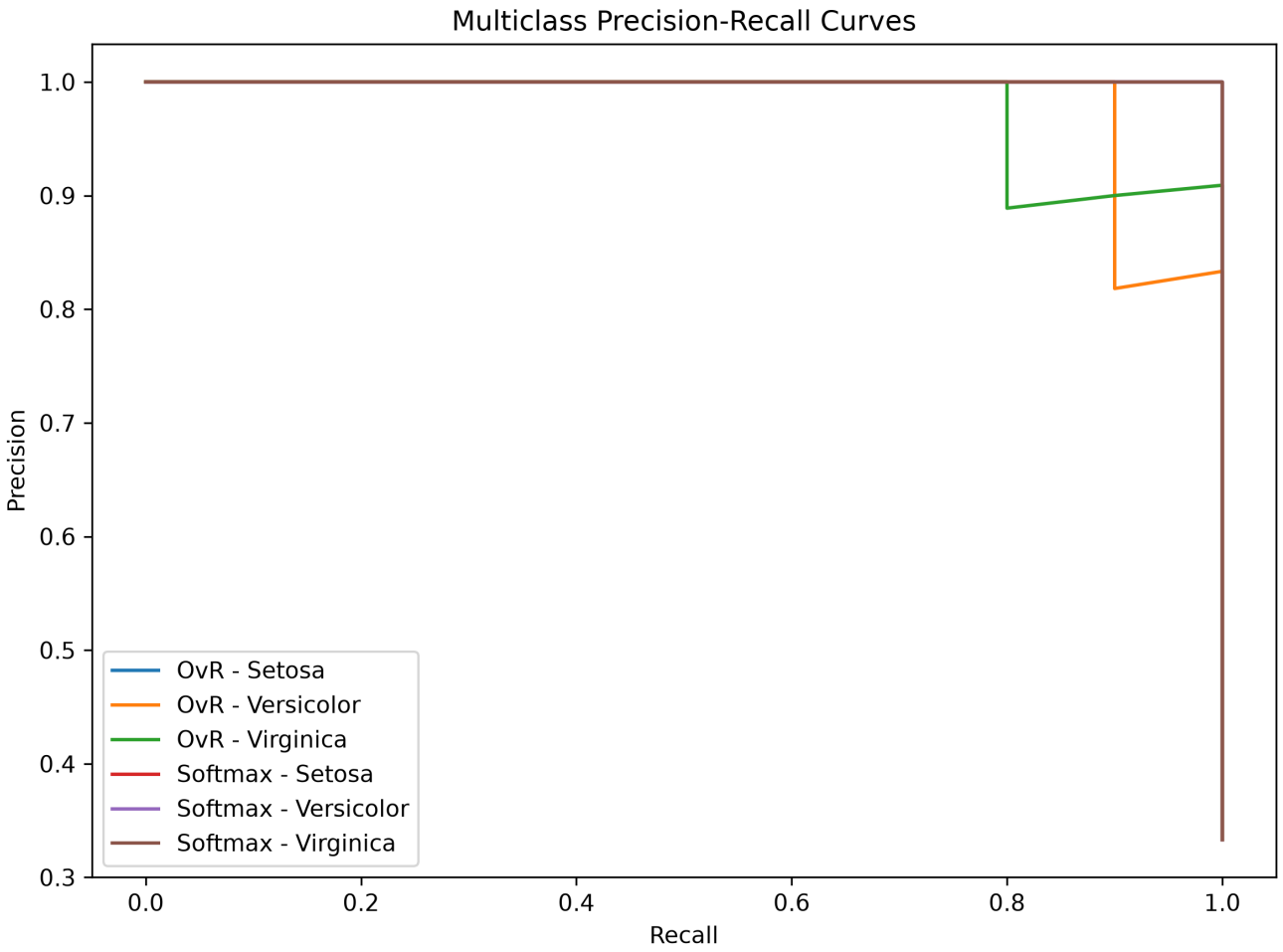


Figure D.15: Multiclass Precision-Recall Curves

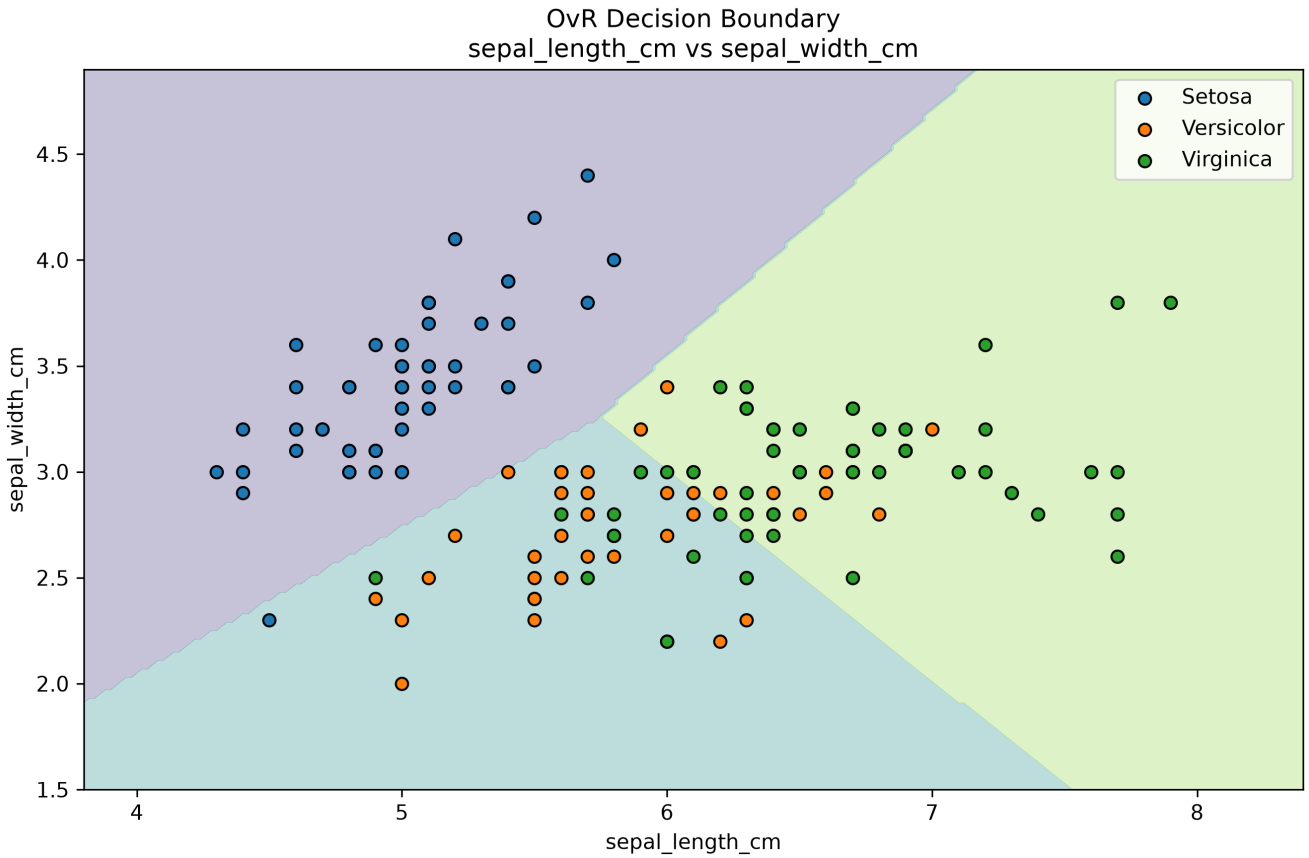


Figure D.16: OvR Decision Boundary: Sepal Features

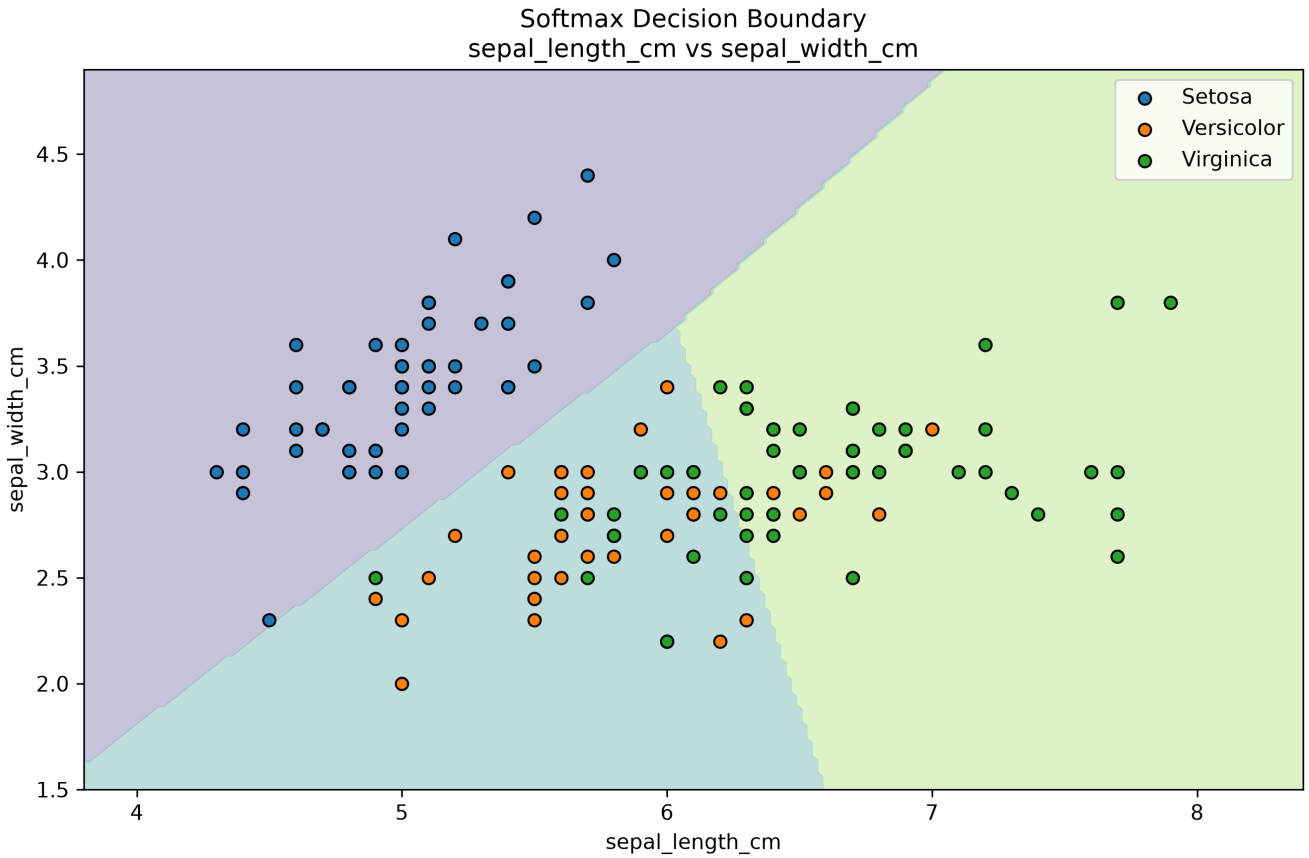


Figure D.17: Softmax Decision Boundary: Sepal Features

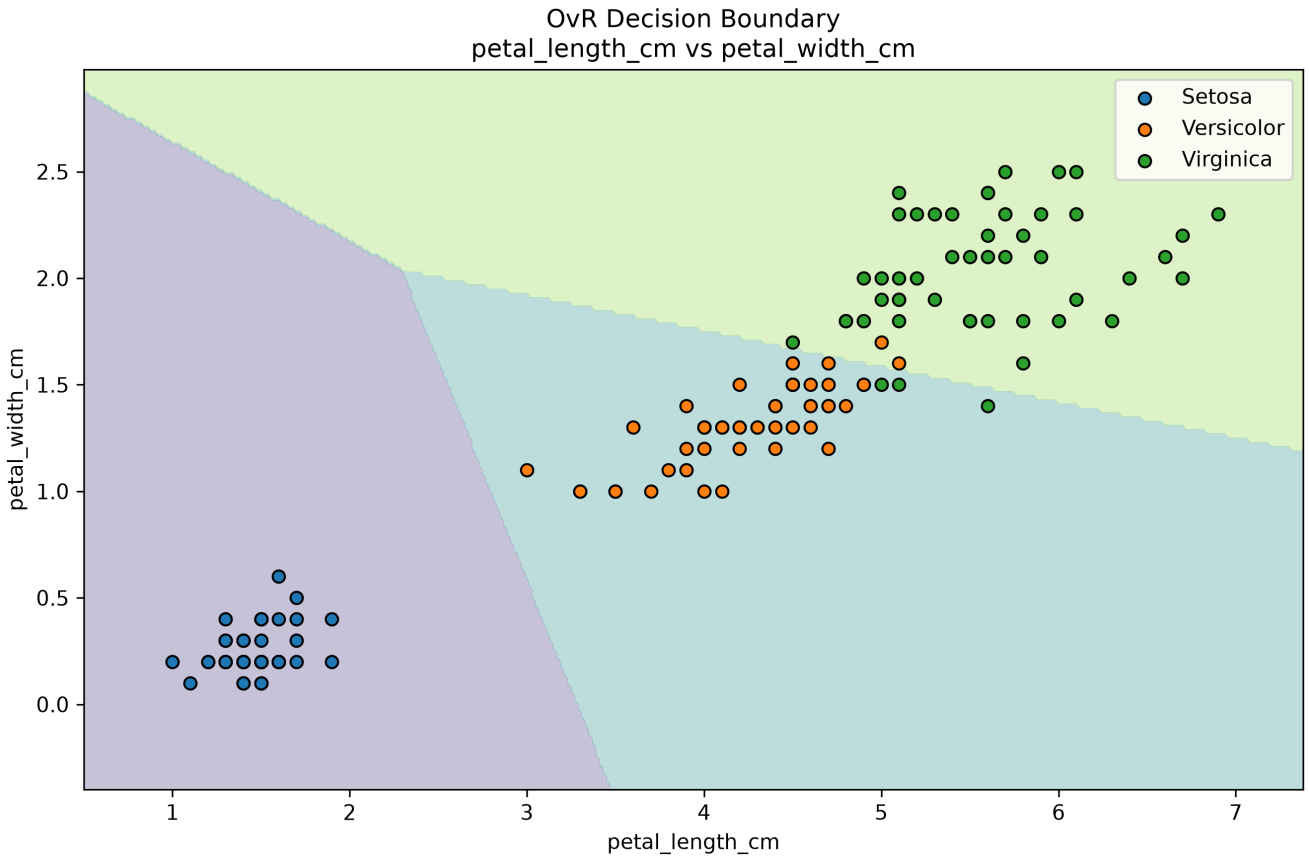


Figure D.18: OvR Decision Boundary: Petal Features

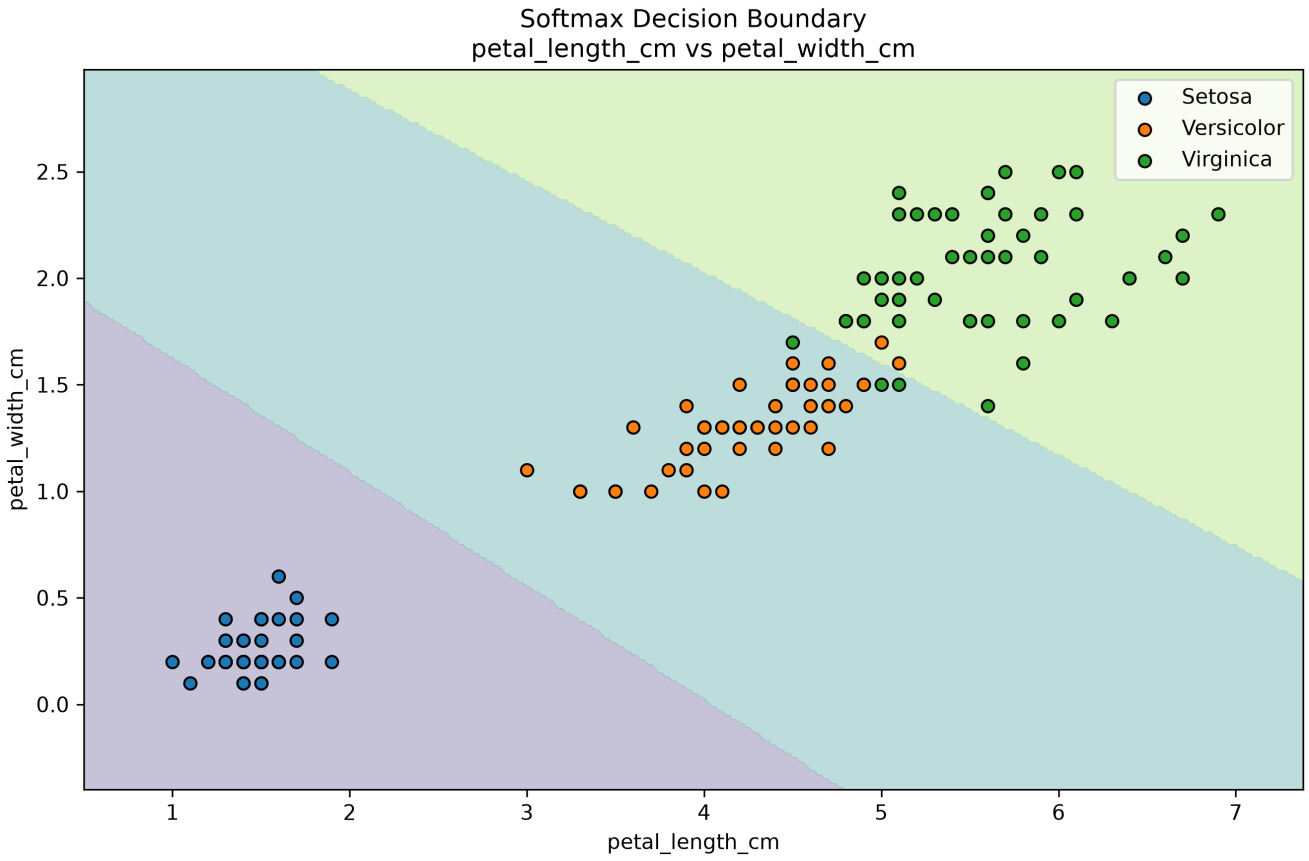


Figure D.19: Softmax Decision Boundary: Petal Features

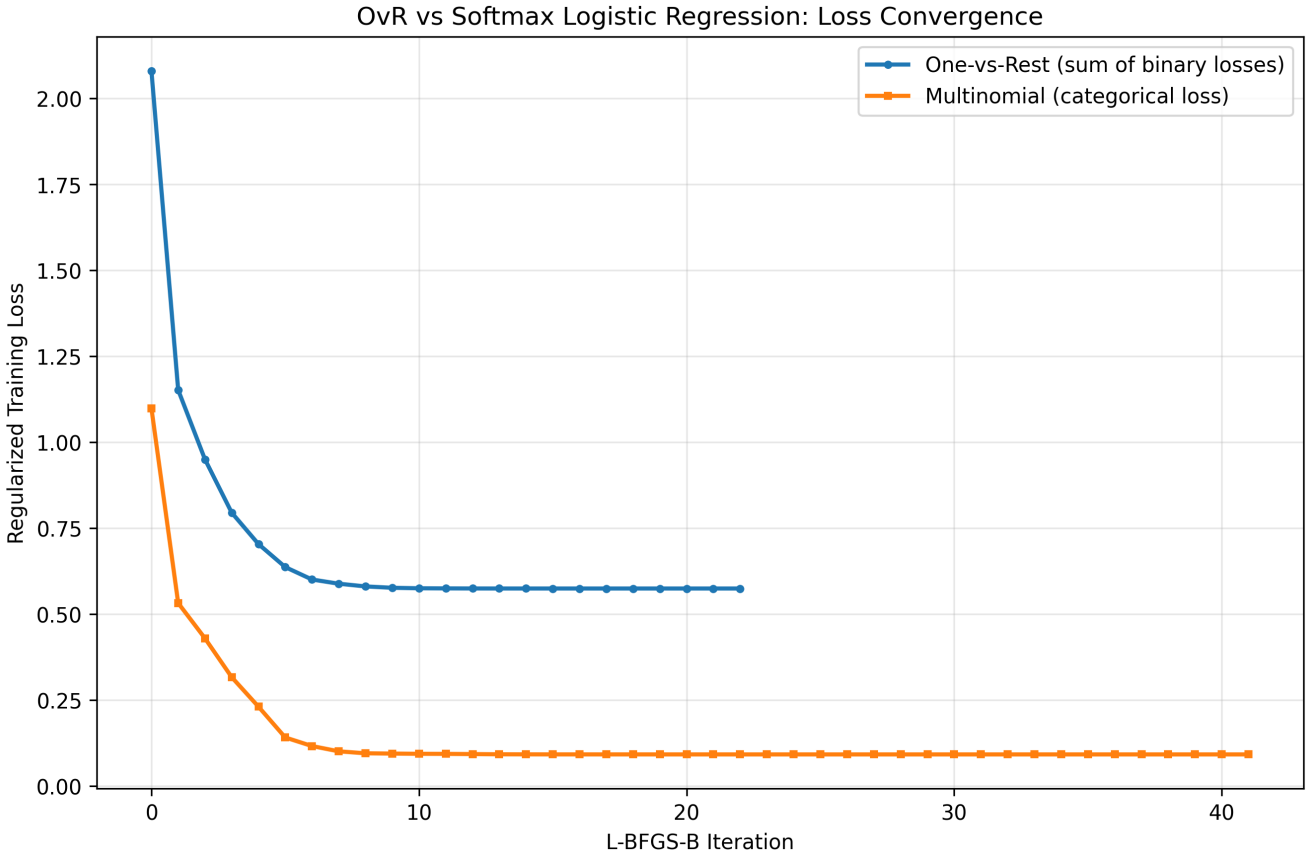


Figure D.20: OvR vs Softmax Loss Convergence

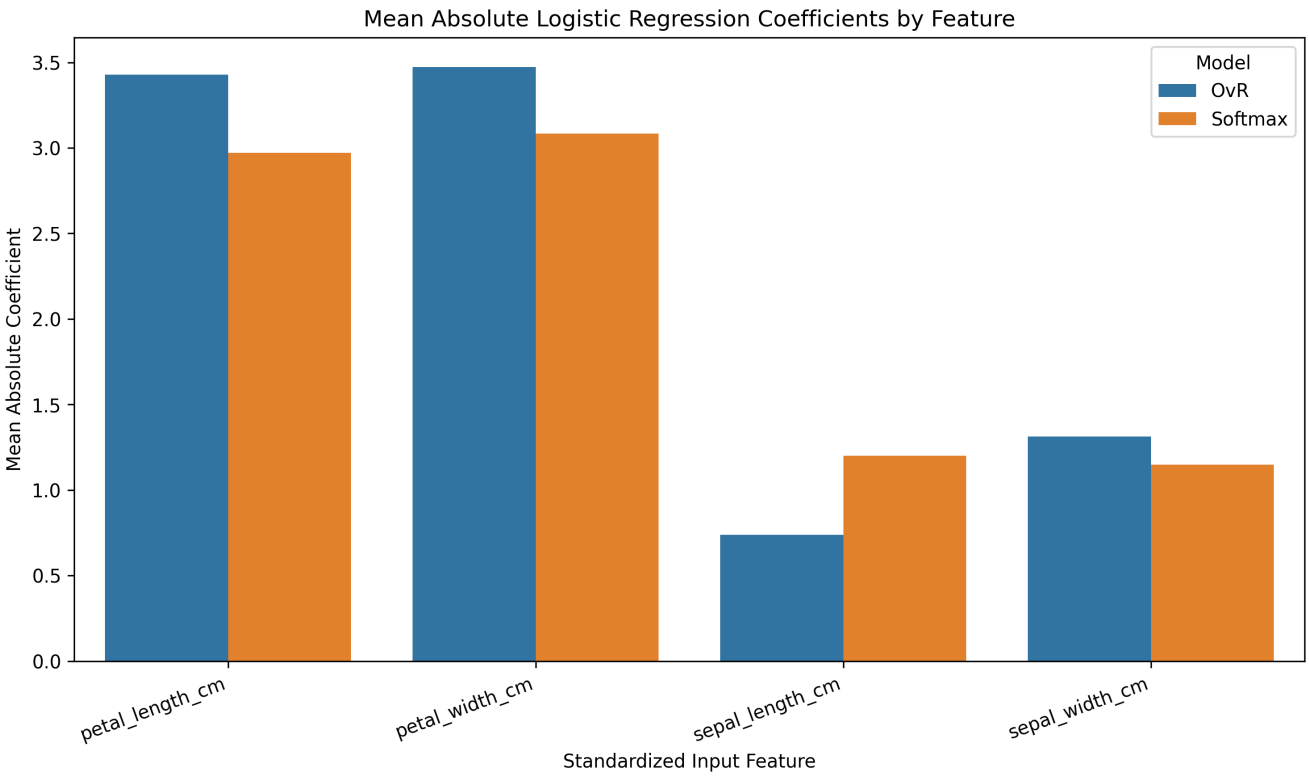


Figure D.21: Mean Absolute Feature Coefficient Comparison

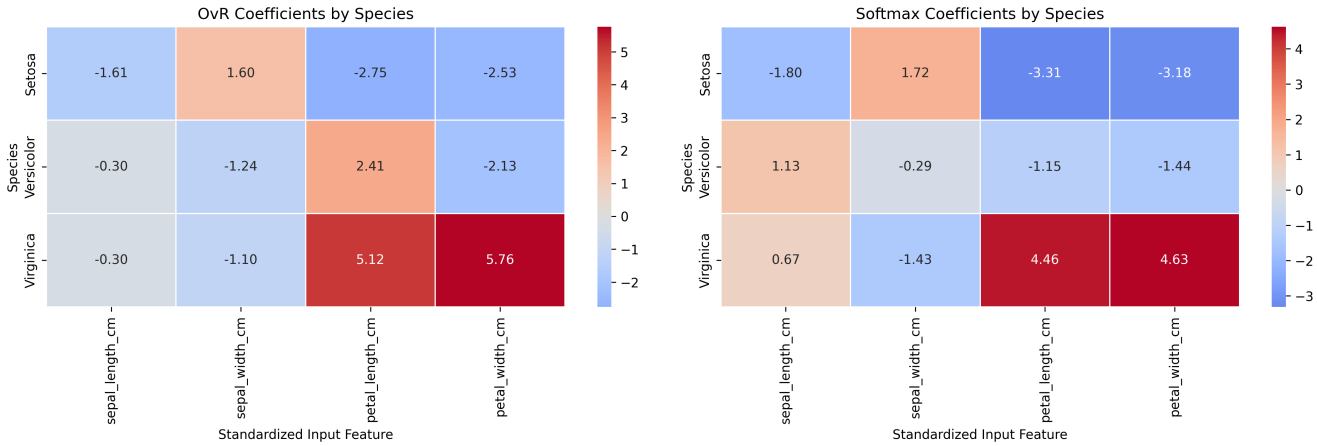


Figure D.22: Class-wise OvR and Softmax Coefficient Heatmaps